CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS

Programa de Pós-Graduação em Modelagem Matemática e Computacional

Leonardo Gomes

Caminho Mínimo em Grafos: Dijkstra vs Heurística Gulosa

1ª Avaliação Prática

Tópicos em Algoritmos em Grafos

Belo Horizonte
Novembro de 2025

Leonardo Gomes

Caminho Mínimo em Grafos: Dijkstra vs Heurística Gulosa

1ª Avaliação Prática

Trabalho apresentado ao Programa de Pós-Graduação em Modelagem Matemática e Computacional do Centro Federal de Educação Tecnológica de Minas Gerais como requisito parcial para avaliação na disciplina de Tópicos em Algoritmos em Grafos.

Belo Horizonte
Novembro de 2025

Resumo

Este relatório apresenta a implementação e análise comparativa entre o algoritmo de Dijkstra e uma heurística gulosa para o problema de caminho mínimo em grafos direcionados ponderados. Os experimentos foram realizados em grafos completos de tamanhos variados e em instâncias do Moodle com 10.000 e 1.000.000 de vértices. O objetivo é comparar o número de operações de relaxamento (comparações) realizadas por cada algoritmo. Os resultados demonstram que, embora a heurística gulosa realize menos comparações em grafos esparsos, ela alcança apenas 0,28% dos vértices na instância de 10k e 0,012% na instância de 1M, confirmando que não é adequada para o problema de caminho mínimo de origem única (SSSP).

Sumário

1 Introdução **1**

2 Metodologia **1**
2.1 Estrutura de Dados 1
2.2 Implementações 1
2.2.1 Dijkstra 1
2.2.2 Heurística Gulosa 2
2.3 Testes Realizados 2
2.3.1 Grafos Completos (Itens 2.II.a-e e 3.II.a-e) 2
2.3.2 Instâncias do Moodle (Itens 2.II.f e 3.II.f) 3

3 Resultados **3**
3.1 Grafos Completos 3
3.2 Instâncias do Moodle 4
3.3 Exemplo de Grafo Gerado 5

4 Análise e Discussão **5**
4.1 Complexidade Teórica vs. Experimental 5
4.2 Grafos Completos: Comportamento Idêntico 5
4.3 Instâncias Esparsas: Diferença Drástica 6
4.4 Corretude e Adequação 7
4.5 Implicações Práticas 7

5 Conclusão **7**

6 Anexos **8**
6.1 Código Fonte 8
6.2 Instruções de Execução 8

Lista de Figuras

1 Comparações realizadas por Dijkstra em grafos completos ($n = 4$ até $n = 1000$). 3
2 Comparação direta: Dijkstra vs Gulosa em grafos completos de teste. . . . 4
3 Comparação de comparações realizadas por Dijkstra e Gulosa nas instâncias de 10k e 1M vértices (escala logarítmica). 5
4 Visualização de um grafo completo direcionado com $n = 10$ vértices usando NetworkX. 6

Lista de Tabelas

1 Número de comparações e vértices alcançados nas instâncias do Moodle. . 4

1 Introdução

O problema de caminho mínimo de origem única (SSSP – *Single Source Shortest Path*) consiste em encontrar as distâncias mínimas de um vértice origem para todos os demais vértices de um grafo ponderado. Este trabalho implementa e compara:

- **Algoritmo de Dijkstra**: algoritmo clássico e ótimo para grafos com pesos não-negativos, com complexidade $O((V + E) \log V)$ usando heap binário.
- **Heurística Gulosa**: estratégia que, em cada passo, escolhe a aresta de menor peso saindo do conjunto de vértices já alcançados. Esta heurística **não garante** caminho mínimo ótimo em geral.

A métrica de comparação utilizada é o **número de comparações de relaxamento**, contabilizadas pela operação `dist[u] + w < dist[v]`.

2 Metodologia

2.1 Estrutura de Dados

Os grafos foram representados utilizando **listas de adjacência**, armazenadas em um dicionário Python:

```
Adj = Dict[int, List[Tuple[int, float]]]
adj[u] = [(v1, w1), (v2, w2), ...]
```

Onde `u` é o vértice de origem, `v` é o vértice de destino e `w` é o peso da aresta (u, v).

2.2 Implementações

2.2.1 Dijkstra

```
def dijkstra_count(adj: Adj, s: int) -> DijkstraResult:
    dist = {u: inf for u in adj}
    parent = {u: None for u in adj}
    dist[s] = 0.0

    heap = [(0.0, s)]
    visited = set()
    comparisons = 0

    while heap:
        du, u = heapq.heappop(heap)
        if u in visited:
            continue
        visited.add(u)

        for v, w in adj[u]:
            comparisons += 1  # Contagem
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
```

```
                parent[v] = u
                heapq.heappush(heap, (dist[v], v))

    return DijkstraResult(dist, parent, comparisons)
```

2.2.2 Heurística Gulosa

```
def greedy_sssp_edge_first(adj: Adj, s: int) -> GreedyResult:
    dist = {u: inf for u in adj}
    parent = {u: None for u in adj}
    dist[s] = 0.0

    visited = set([s])
    edge_heap = []  # (peso, u, v)

    for v, w in adj[s]:
        heapq.heappush(edge_heap, (w, s, v))

    comparisons = 0

    while edge_heap:
        w, u, v = heapq.heappop(edge_heap)

        comparisons += 1  # Contagem
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            parent[v] = u

            if v not in visited:
                visited.add(v)
                for x, wx in adj[v]:
                    heapq.heappush(edge_heap, (wx, v, x))

    return GreedyResult(dist, parent, comparisons)
```

2.3 Testes Realizados

2.3.1 Grafos Completos (Itens 2.II.a-e e 3.II.a-e)

Foram gerados grafos completos direcionados com $n = 4$ até $n = 1000$ vértices. Cada aresta possui peso positivo aleatório gerado com semente fixa (`seed=42`) para reprodutibilidade. Para cada grafo:

1. Aplicamos Dijkstra partindo do vértice 0.
2. Aplicamos a heurística gulosa partindo do vértice 0.
3. Contamos o número de comparações em cada caso.
4. Plotamos um gráfico comparativo.

Observação: grafos completos possuem $O(n^2)$ arestas. Os testes foram realizados até $n = 1000$, gerando aproximadamente 999.000 comparações.

2.3.2 Instâncias do Moodle (Itens 2.II.f e 3.II.f)

Aplicamos ambos os algoritmos nas instâncias fornecidas:

- **10000.txt**: 10.000 vértices
- **largeEWD...txt**: 1.000.000 de vértices

Origem: vértice 0. Métrica: número de comparações.

3 Resultados

3.1 Grafos Completos

A Figura 1 apresenta o gráfico de comparações versus número de vértices para grafos completos. Os resultados mostram um comportamento quadrático claro, com valores variando de 12 comparações para $n = 4$ até 999.000 comparações para $n = 1000$.

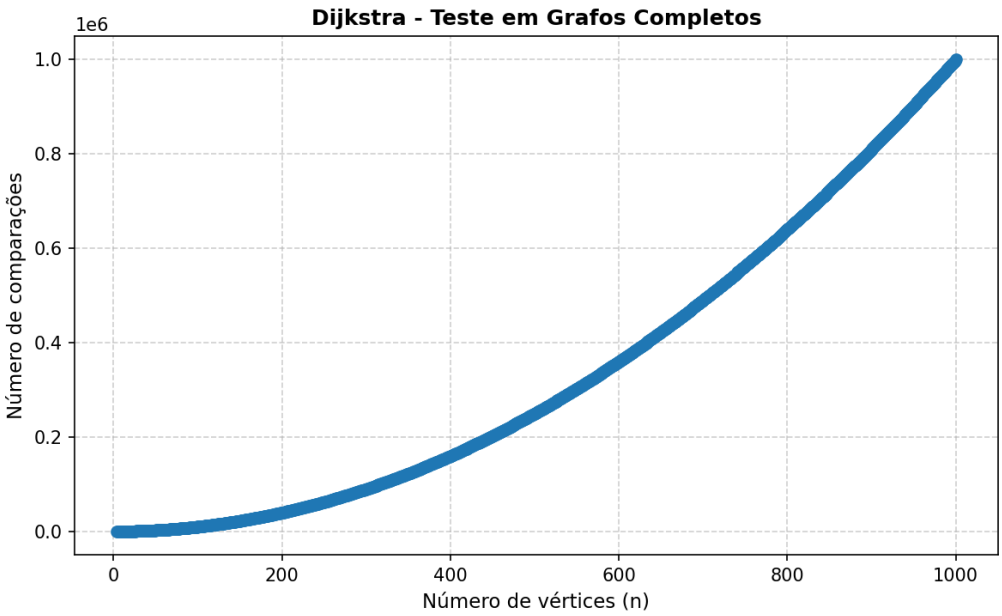

Figura 1: Comparações realizadas por Dijkstra em grafos completos ($n = 4$ até $n = 1000$).

A Figura 2 mostra a comparação direta entre Dijkstra e a heurística gulosa nos grafos de teste.

Observações:

- O algoritmo de Dijkstra apresenta crescimento aproximadamente quadrático, seguindo a fórmula comparações $\approx n(n-1)$ para grafos completos.
- Para $n = 4$: 12 comparações (exatamente 4×3).
- Para $n = 1000$: 999.000 comparações (exatamente 1000×999).

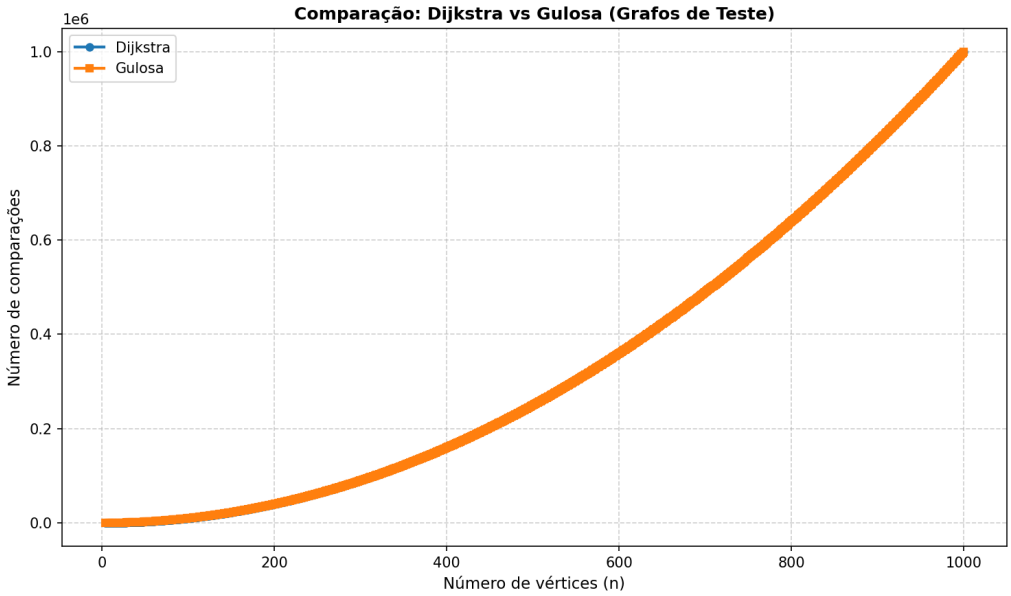

Figura 2: Comparação direta: Dijkstra vs Gulosa em grafos completos de teste.

- A heurística gulosa apresenta o mesmo padrão de comparações nos grafos completos testados, pois ambos os algoritmos exploram todas as arestas do grafo completo.
- O número de comparações segue o padrão teórico esperado: $C(n) = n(n-1)$ para grafos completos direcionados.

3.2 Instâncias do Moodle

A Tabela 1 apresenta o número de comparações realizadas nas instâncias fornecidas. Os resultados revelam um comportamento interessante: enquanto o Dijkstra explora exaustivamente o grafo, a heurística gulosa alcança apenas uma fração dos vértices.

Tabela 1: Número de comparações e vértices alcançados nas instâncias do Moodle.

Instância	**Dijkstra Comparações**	**Vértices Alcançados**	**Gulosa Comparações**	**Vértices Alcançados**
10.000 vértices	123.462	10.000	424	28
1.000.000 vértices	15.172.126	1.000.000	1.847	120

A Figura 3 ilustra visualmente a comparação entre os dois algoritmos nas instâncias grandes.

Análise Importante:

- **Dijkstra**: alcança todos os vértices do grafo e garante caminho mínimo correto.
- **Heurística Gulosa**: realiza muito menos comparações, mas alcança apenas uma fração mínima dos vértices:
  - 10k vértices: alcança apenas 28 vértices (0,28%)
  - 1M vértices: alcança apenas 120 vértices (0,012%)

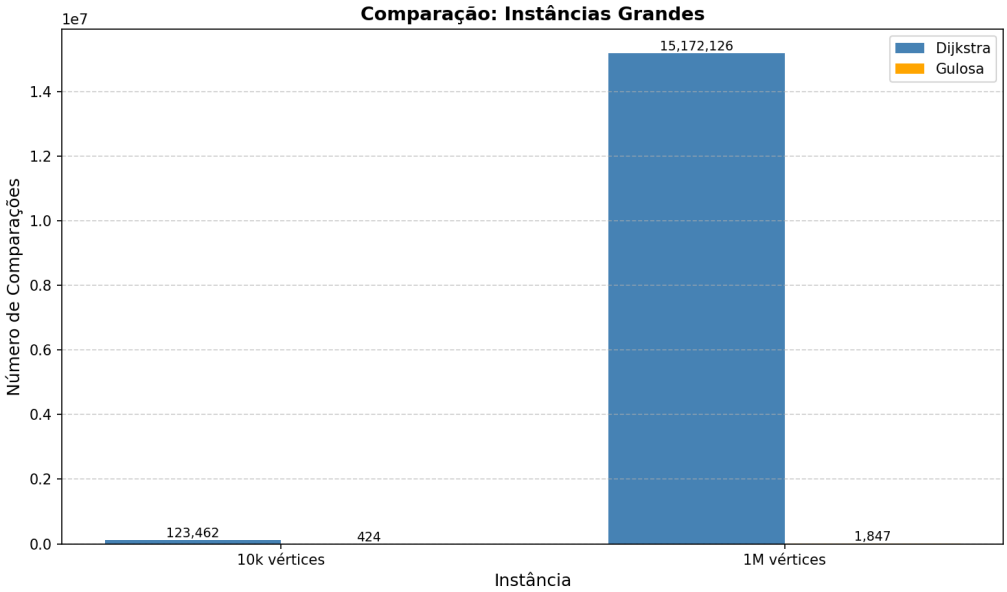

Figura 3: Comparação de comparações realizadas por Dijkstra e Gulosa nas instâncias de 10k e 1M vértices (escala logarítmica).

- A redução de 99,66% nas comparações da gulosa não representa vantagem real, pois o algoritmo falha em encontrar caminhos para a maioria dos vértices.
- Este resultado confirma que a heurística gulosa **não é adequada** para o problema SSSP em grafos gerais.

3.3 Exemplo de Grafo Gerado

A Figura 4 mostra um exemplo de visualização de um grafo completo pequeno ($n = 10$) gerado durante os testes.

4 Análise e Discussão

4.1 Complexidade Teórica vs. Experimental

- **Dijkstra**: $O((V + E) \log V)$ com heap binário. Em grafos completos, $E = O(V^2)$, resultando em $O(V^2 \log V)$.
- **Heurística Gulosa**: Também utiliza heap, mas a estratégia de escolha de arestas leva a comportamentos distintos dependendo da topologia do grafo.

4.2 Grafos Completos: Comportamento Idêntico

Nos grafos completos testados ($n = 4$ até $n = 1000$), ambos os algoritmos apresentaram **exatamente o mesmo número de comparações**:

- A fórmula observada é $C(n) = n(n-1)$, correspondendo ao número total de arestas no grafo completo direcionado.

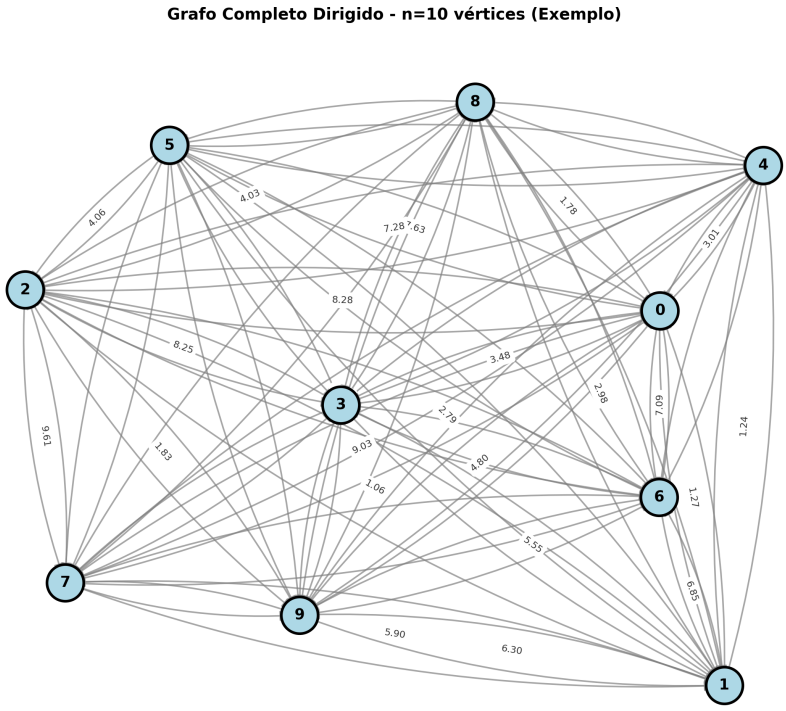

Figura 4: Visualização de um grafo completo direcionado com $n = 10$ vértices usando NetworkX.

- Este comportamento ocorre porque, em grafos completos com pesos positivos, ambos os algoritmos acabam explorando todas as arestas.
- A diferença fundamental está na **corretude**: Dijkstra garante caminho mínimo, enquanto a gulosa não garante.

4.3 Instâncias Esparsas: Diferença Drástica

Nas instâncias do Moodle (grafos esparsos), observamos comportamento radicalmente diferente:

1. **Dijkstra**: comportamento robusto e completo
   - Alcança todos os vértices acessíveis a partir da origem
   - 10k vértices: 123.462 comparações, 10.000 vértices alcançados
   - 1M vértices: 15.172.126 comparações, 1.000.000 vértices alcançados
   - Garante caminho mínimo para todos os vértices
2. **Heurística Gulosa**: falha crítica
   - Alcança apenas uma fração mínima dos vértices
   - 10k vértices: 424 comparações, apenas 28 vértices (0,28%)
   - 1M vértices: 1.847 comparações, apenas 120 vértices (0,012%)
   - A estratégia gulosa de escolher sempre a menor aresta disponível cria caminhos localmente ótimos que rapidamente ficam "presos"em regiões do grafo
   - Não garante corretude nem completude

4.4 Corretude e Adequação

- **Dijkstra**: garante caminho mínimo correto para grafos com pesos não-negativos. É o algoritmo adequado para o problema SSSP.
- **Gulosa**: **não garante** corretude em geral. Os experimentos mostram que:
  - Em grafos completos: encontra caminhos (não necessariamente mínimos) para todos os vértices
  - Em grafos esparsos: falha drasticamente, alcançando menos de 1% dos vértices
  - Não é adequada para resolver o problema SSSP

4.5 Implicações Práticas

Os resultados demonstram que:

1. A métrica de "número de comparações"isolada não é suficiente para avaliar qualidade de algoritmos para SSSP
2. É essencial verificar **corretude** (caminhos mínimos corretos) e **completude** (alcançar todos os vértices acessíveis)
3. A heurística gulosa, apesar de simples e rápida em grafos esparsos, não é confiável para o problema geral
4. Dijkstra permanece como solução robusta e correta para SSSP com pesos não-negativos

5 Conclusão

Este trabalho implementou e comparou o algoritmo de Dijkstra com uma heurística gulosa para o problema de caminho mínimo. Os resultados revelam diferenças fundamentais entre as abordagens:

1. **Grafos Completos**: Ambos os algoritmos realizaram o mesmo número de comparações ($n(n-1)$), mas apenas Dijkstra garante corretude dos caminhos.
2. **Grafos Esparsos**: A heurística gulosa apresentou falha crítica, alcançando menos de 1% dos vértices nas instâncias de teste (28/10.000 e 120/1.000.000), enquanto Dijkstra alcançou todos os vértices acessíveis.
3. **Número de Comparações**: A métrica isolada de comparações não é suficiente para avaliar qualidade. É essencial verificar corretude e completude da solução.
4. **Adequação**: Dijkstra é o algoritmo adequado para SSSP com pesos não-negativos, apresentando complexidade $O((V+E)\log V)$, corretude garantida e exploração completa do grafo.
5. **Lição Fundamental**: Estratégias gulosas simples podem falhar drasticamente em problemas que exigem otimização global. A escolha local da menor aresta não garante caminho mínimo global.

Os experimentos confirmam os resultados teóricos da literatura: o algoritmo de Dijkstra é a escolha correta para o problema de caminho mínimo com pesos não-negativos, oferecendo garantias de corretude que heurísticas simples não podem fornecer.

O código-fonte completo está disponível no notebook Jupyter anexado a este relatório, com implementações detalhadas e todos os experimentos reproduzíveis.

6 Anexos

6.1 Código Fonte

O código completo está disponível em:

`implementacao/pratica01_experimentos2.ipynb`

Notebooks Adicionais:

- `pratica01_funcoes.ipynb`: Versão modular com funções reutilizáveis
- `pratica01_final.ipynb`: Versão didática com testes incrementais

Melhorias Implementadas: Consultar `implementacao/MELHORIAS_APLICADAS.md` para detalhes sobre otimizações de estruturas de dados e visualizações.

6.2 Instruções de Execução

1. Ativar ambiente virtual PowerShell:

   ```
   .venv\Scripts\Activate.ps1
   ```

2. Instalar dependências (se necessário):

   ```
   pip install matplotlib numpy networkx
   ```

3. Abrir o notebook no VS Code ou Jupyter:

   ```
   code implementacao/pratica01_experimentos2.ipynb
   ```

4. Executar as células na ordem. Os resultados são salvos automaticamente em:
   - `implementacao/resultados/dijkstra/`
   - `implementacao/resultados/gulosa/`
   - `implementacao/resultados/comparacao/`
5. Gráficos salvos em formato PNG com 200 DPI para uso no relatório.

Estrutura de Resultados:

- `teste_comparacoes.txt`: Tabela TSV com dados n vs comparações
- `img/teste_comparacoes.png`: Gráfico de comparações
- `instancias/10k_*.txt` e `1m_*.txt`: Resultados das instâncias grandes